Project-Proposal

KINDERGARDEN - MANAGEMENTSYSTEM

TUHH

Technische Universität Hamburg

Projectteam 13
Technische Universität Hamburg – SoSe 2025
Submission Date: 04.07.2025
Scheduling System / Favorite Meal System

Contents

1 INTRODUCTION 2

2 GOALS & MOTIVATION 3

3 PROJECT ORGANIZATION & WORKING METHODOLOGY 3

4 REQUIREMENTS & USER STORIES 3

5 ARCHITECTURE & SYSTEM OVERVIEW 4

6 MODEL DESCRIPTIONS 4

6.1 USE-CASE-DIAGRAMM 5
6.2 CLASS-DIAGRAM 6
6.3 ACTIVITY-DIAGRAMM 6
6.4 EXTENDED FINITE STATE MACHINES (EFSM) 8
6.5 COMMUNICATION-DIAGRAMM 9
6.6 SEQUENCE-DIAGRAMM 10
6.7 TIME PETRI NETS (TAPN) 10

7 FORMALE SPEZIFIKATION / OCL PRE- & POSTCONDITIONS / OCL INVARIANTS 11

8 TEST DOCUMENTATION & QUALITY ASSURANCE 13

8.1 CODE-BASED TESTSUITE 13

8.2 DOMAIN-BASED TESTSUITE 14

9 LESSONS LEARNED 15

10 ATTACHMENTS 15

1 Introduction

The daycare management system is a web-based solution that digitizes the daily organizational tasks of daycare centers. It handles staff vacation and sick leave notifications, children's meal coordination, as well as staff and group planning. All stakeholders – management, teaching staff, kitchen team, parents, and children – are provided with a centralized platform that promotes transparency, efficiency, and satisfaction.

2 Goals & Motivation

- Reduction of paperwork and administrative effort through digitalized workflows
- Automated notifications for staff shortages and coordinated meal plans
- Intuitive usability for all user groups (tablet-optimized UI)
- Traceability of all processes via versioning and logging
- Extendable architecture for future modules (e.g., billing, parent chat)

3 Project Organization & Working Methodology

- Methodology: Scrum (four sprints)
- Collaboration tools: GitLab (version control), Discord (communication)
- Tasks were strategically and equitably distributed across the team.
- Multiple team members repeatedly reviewed and refined most models.
- This collaborative process ensured exceptional quality, traceability, and consistency across all models and documentation.
- Iterative modeling: Each model iteration was validated against user stories in review sessions.

4 Requirements & User Stories

• Digital submission and management of vacation requests and sick leave notifications

• Regular meal voting by children via tablets

• Automated detection and alerts for staff shortages

• Modular, extensible system architecture

• Comprehensive, auditable documentation of all models and processes

5 Architecture & System Overview

Modules:

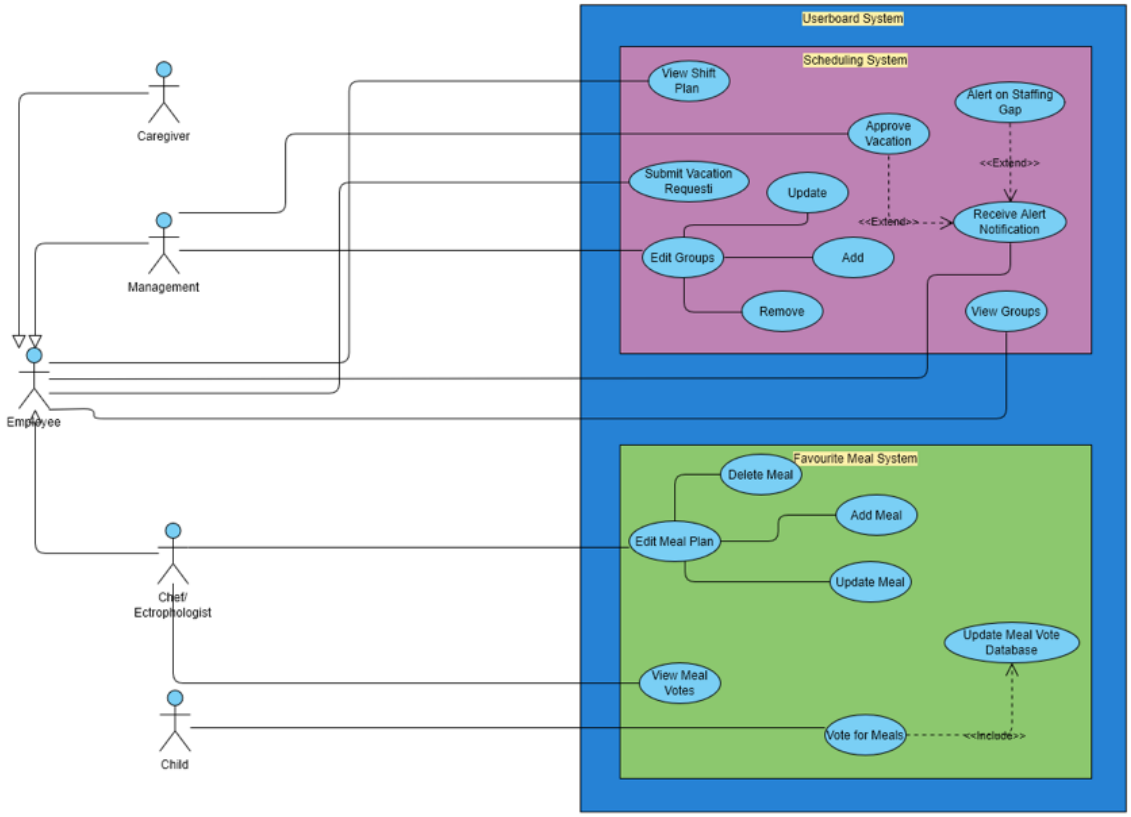
- Scheduling System – Vacation/sick leave management and shift planning
- Favorite Meal System – Meal voting and meal plan management
- Dashboard – Central overview, notifications, and management functions
- External Interfaces – e,g., integration with HR or billing systems

Architecture:

Component-based design supporting parallel processing and real-time communication (REST APIs + WebSockets).

6 Model Descriptions

6.1 Use-Case-Diagramm



The use case diagram illustrates the organization of the daycare system which comprises two main subsystems: the Scheduling System and the Favorite Meal System. The actors Management, Caregiver and Chef inherit from the Employee actor, indicating they share common behavior but also may have distinct responsibilities.

The Scheduling System is designed to manage staff availability by organizing shift plans and handling vacation requests. It also supports group management for better structure and coordination. If a staffing gap arises, the system sends alert notifications to ensure timely intervention by management.

The favorite meal system allows children to vote for their favorite meals respectively what they want to eat in that week. Furthermore, the chefs can adjust the meal plan, i.e. delete, add, or update meals. This ensures variation in the meal plan, where seasonal ingredients can be considered. This approach supports nutritional variety.

Assumptions:

- User-Accounts already exist

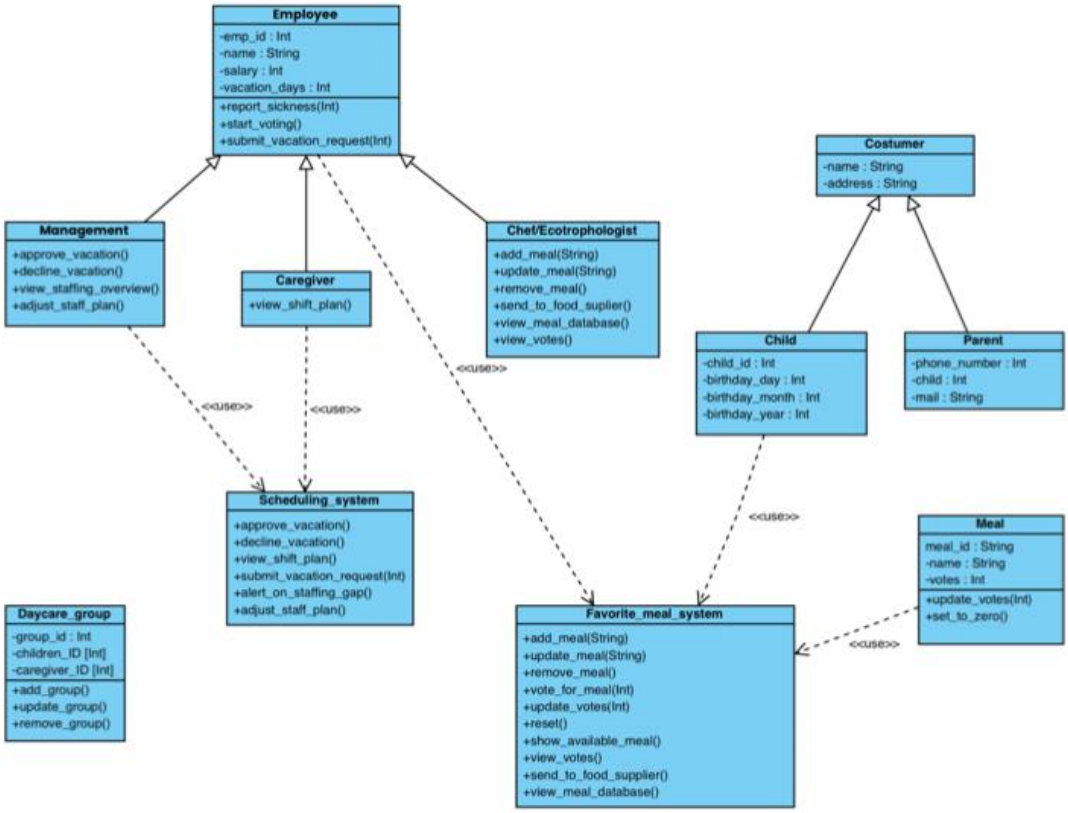
- every staff member has a device on which the system runs

- Predefined meal options

- Management already planned staff schedule and organization is set up, groups created and assigned

- App is already installed and runnable on devices and bug free

6.2 Class-Diagram

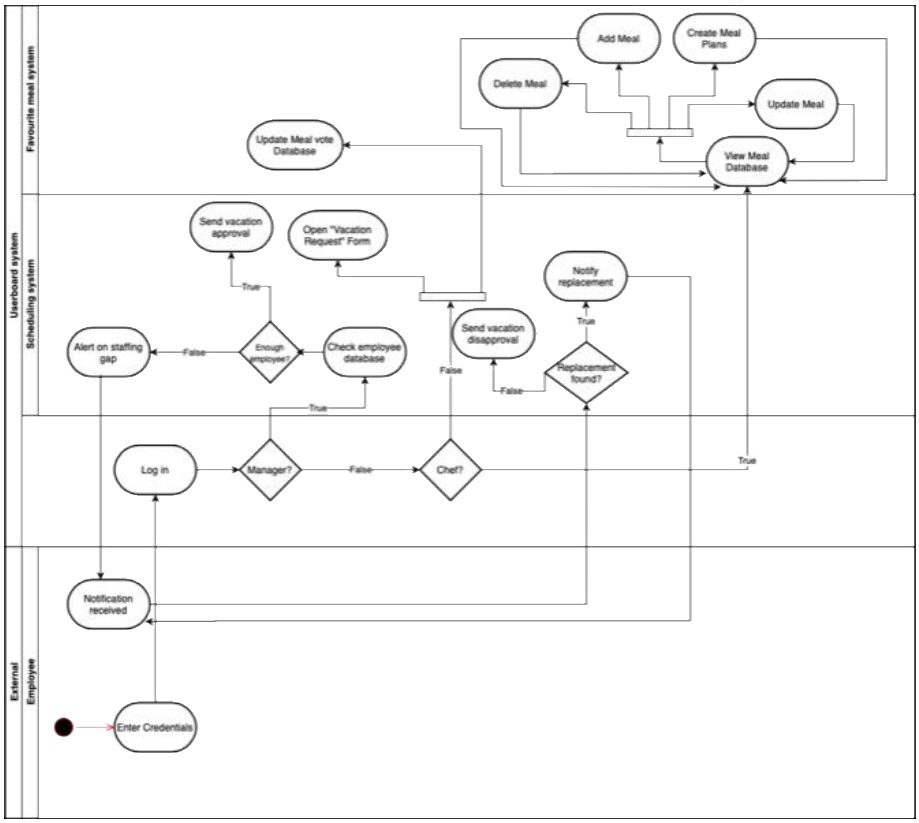


The management, the caregiver and the chef classes are inherited by the employee class and have access to all private attributes of employee.

Employees use the Scheduling System for staff-planning, i.e. absence due to illness or vacation planning. The chef/ecotrophologist uses the favorite food system to add or update the dishes.

The child- and parent class are inherited by the customer class. The child class uses the favorite food system to vote for the meal they want to eat in that week.

6.3 Activity-Diagramm



The activity diagram models the dynamic workflows of the daycare system and illustrates how users and system components interact across two main functional areas: Favorite Meal and Scheduling System. The diagram captures both sequential and parallel activities and is structured using swimlanes to clearly assign responsibilities.

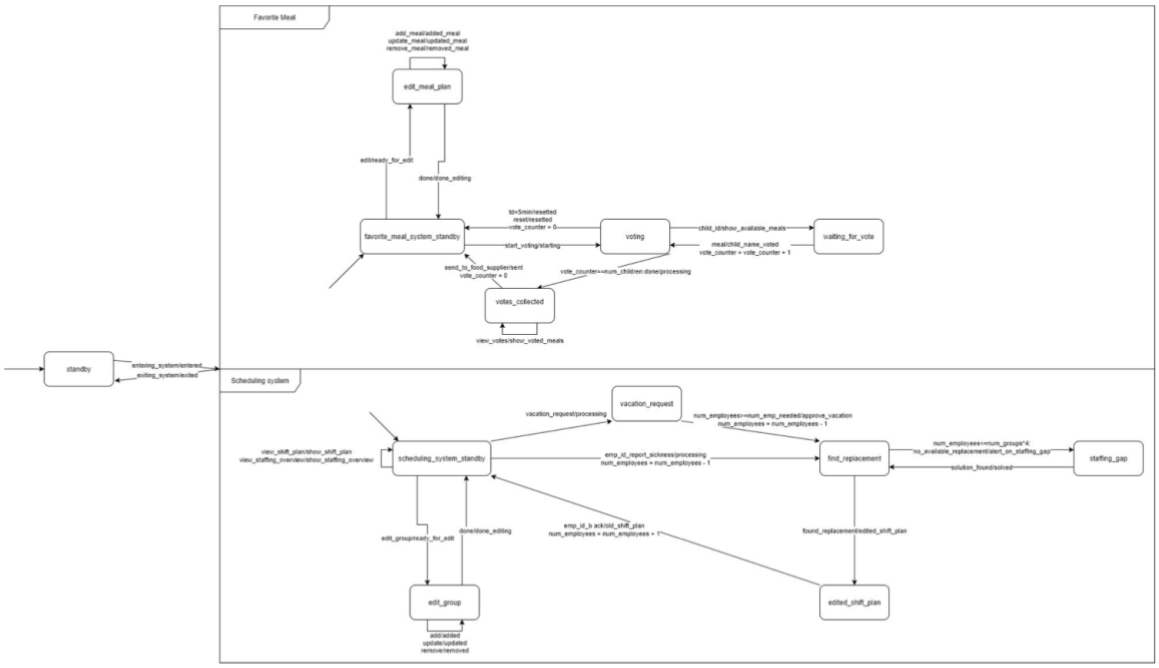
In the Favorite Meal section, the diagram shows how meals are managed - from creating and editing the weekly meal plan to initiating a voting process where children select their preferred meals. The process includes handling user input, counting votes, and sending the final result to the food supplier.

The Scheduling System section visualizes processes such as requesting vacation, reporting sickness, editing shift plans, and managing staffing shortages. It shows how the system reacts to absences, searches for replacements, and updates schedules accordingly.

Decision and merge nodes are used to control the flow based on conditions such as vote completion or staff availability. Fork and join nodes represent parallel processing where appropriate.

Overall, the activity diagram provides a structured overview of the main processes within the system, how they interact, and how different roles contribute to the daycare's daily operations.

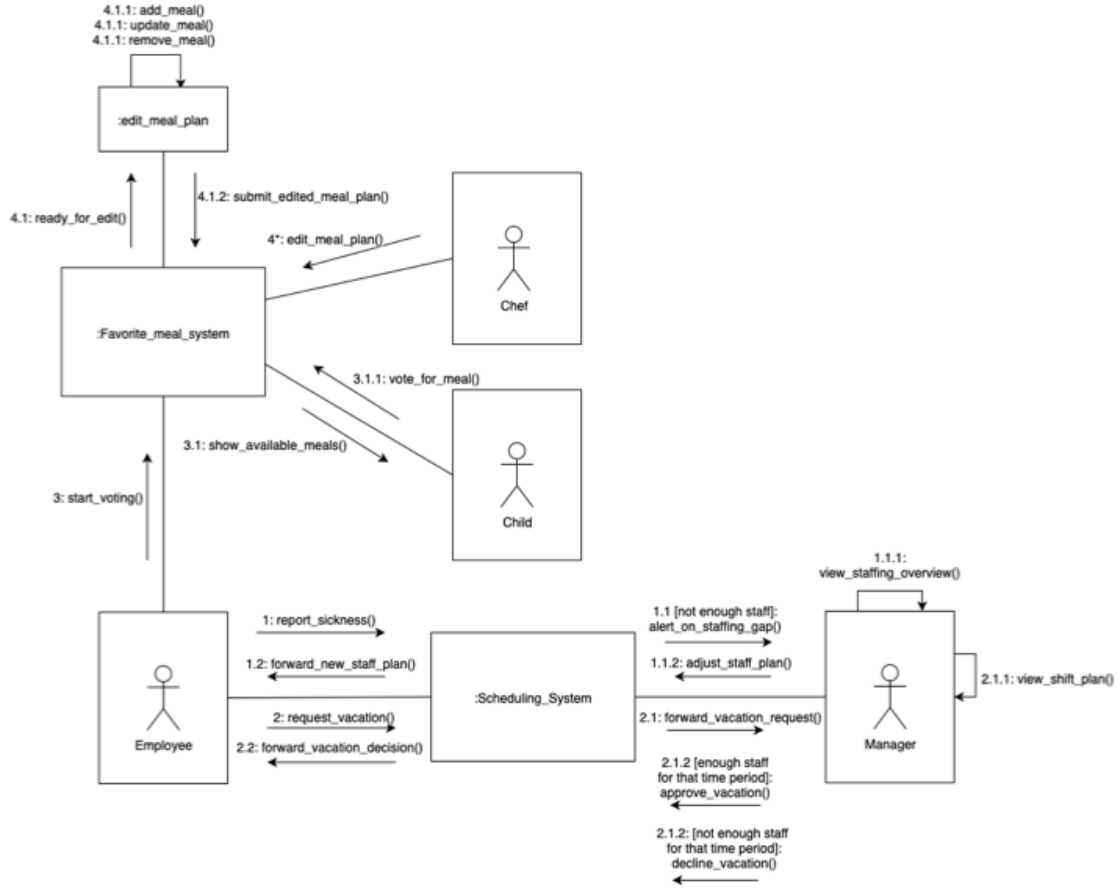
6.4 Extended Finite State Machines (EFSM)



The model consists of two parallel extended finite-state machines: one governing the favorite meal voting process and the other managing shift scheduling and replacements, each rooted in a shared standby state. In the favorite-meal subsystem, the system rests in the “favorite_meal_system_standby” state awaiting either an “edit/ready_for_edit” event, which transitions it to an “edit_meal_plan” state where meals can be added, updated or removed until a “done/done_editing” event returns it to standby, or a “start_voting/starting” event that resets a vote counter and enters a “voting” state; here child actors receive “child_id/show_available_meals,” cast votes via “meal/child_name_voted” which increment the counter, and once the counter equals the number of children, a “vote_counter==num_children/done/processing” transition moves the system to “votes_collected,” where “view_votes/show_voted_meals” may be invoked before a final “send_to_food_supplier/sent” transition returns to standby. Concurrently, the scheduling subsystem remains in “scheduling_system_standby” until it receives “view_shift_plan/show_shift_plan” or “view_staffing_overview/show_staffing_overview” inputs, which merely display information, or “edit_group/ready_for_edit” to enter an “edit_group” state for adding, updating, or removing staff and then return on “done/done_editing.” From standby, “vacation_request/processing” triggers a “vacation_request” state that, when approved, decrements the employee count and returns, while “emp_id_report_sickness/processing” similarly reduces staff and invokes a “find_replacement” operation; in “find_replacement,” if the remaining employees meet or exceed the required threshold a “solution_found/solved” transition produces an “edited_shift_plan” and increases the count back, but if “num_employees<num_groups*4” leads to “no_available_replacement/alert_on_staffing_gap,” the system enters the

“staffing_gap” state until new staffing actions resolve it, at which point a “solution_found/solved” returns control to the standby state.

6.5 Communication-Diagramm



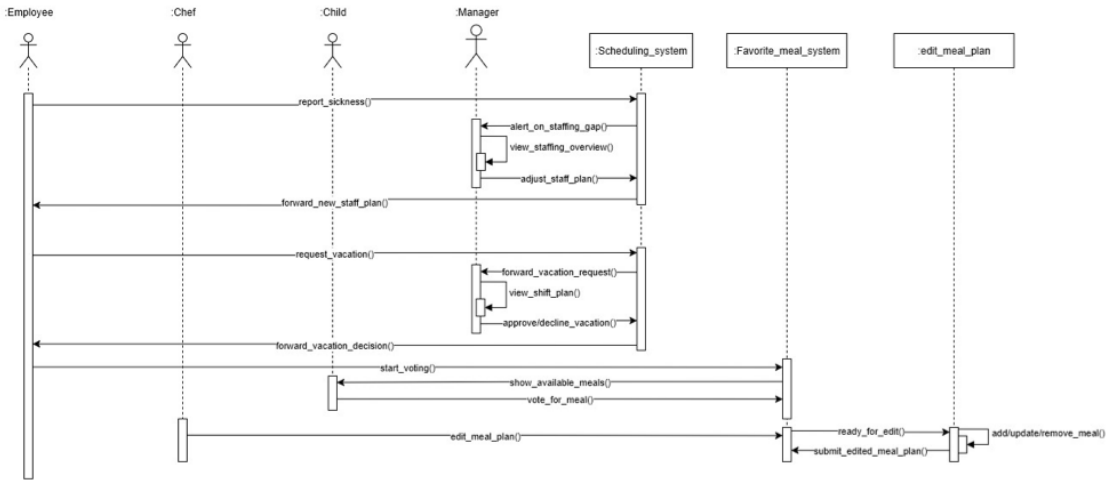
The communication diagram captures the interactions between the users and the system. An employee can report sickness via their device to the scheduling system. If this results in a staffing gap, the system automatically sends an alert to management. The manager, who has access to an overview of all staff schedules, can then adjust the shift plan accordingly. The updated shift plan is forwarded to all affected employees.

When an employee submits a vacation request, it is transmitted through the system to the manager. The manager then reviews the request in the context of the current shift plan and may approve or decline it. Once a decision is made, the Scheduling System notifies the employee of the outcome.

Furthermore, an employee can initiate the favorite meal voting process, which triggers the system to show the available meals as pictures. The children can then choose and vote for a meal.

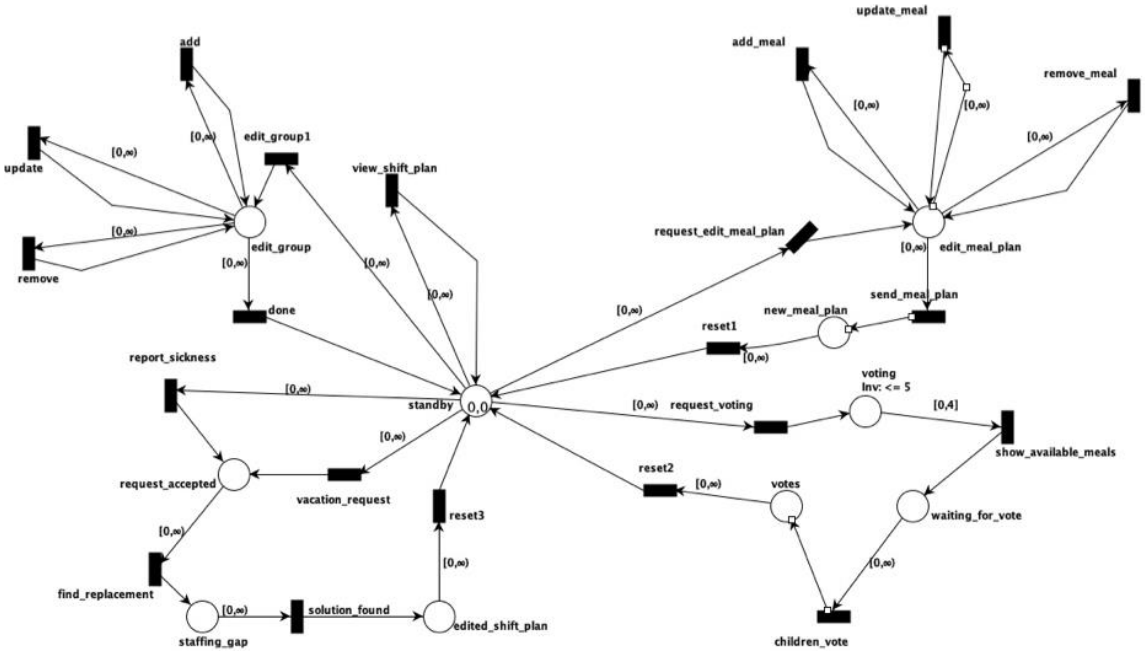
Finally, chefs interact with the system if they want to modify the meal plan. After signaling their intent to edit the plan, they can add, update, or remove meals. These changes are then reflected within the system for voting and planning purposes.

6.6 Sequence-Diagramm



In this model, employees interact with a centralized scheduling system to manage both unexpected absences and planned time off. When an employee reports sickness, the scheduling system immediately alerts the manager to any staffing gaps, who then reviews the current staffing overview, adjusts the staff plan as needed, and forwards the updated plan back to the employees; similarly, when an employee submits a vacation request, the manager receives the forwarded request, examines the shift plan, approves or declines the request, and communicates the decision to the employee(s). Concurrently, the employees can initiate a meal-selection workflow by invoking the favorite meal system to start a voting session, which presents available meal options to the child, records the child's vote, and, once voting concludes, signals the chef that the meal plan is ready for editing. The chef then uses the dedicated meal-plan editing system to add, update, or remove meals and, upon completion, submits the revised meal plan back to the favorite meal system for finalization.

6.7 Time Petri Nets (TAPN)



The TAPN model formalizes the time-driven and parallel behavior of the daycare center support system. It captures core subsystems such as meal voting and shift scheduling, using places for system states and transitions for events, enriched with time constraints and logical conditions.

In the Favorite Meal section, states like favorite_meal_system_standby, edit_meal_plan, voting, waiting_for_vote, and votes_collected reflect the stages of meal planning and voting. Transitions such as add_meal, start_voting, and reset are time-bound—for example, voting is limited to a fixed duration (e.g., $t \in [0,5]$) before automatically resetting. This ensures voting happens within a defined window and cannot be extended indefinitely.

In the Scheduling System, places like scheduling_system_standby, edit_group, vacation_request, find_replacement, edited_shift_plan, and staffing_gap represent the flow of staff planning. Transitions handle actions like processing sick reports, approving vacation, and resolving staff shortages. The model reacts to constraints such as available employee count and automatically triggers escalation if no replacement is found.

The TAPN ensures that both systems progress independently but synchronously, respect deadlines, avoid conflicts, and recover from idle or blocking states through timed resets and escalation paths.

7 Formale Spezifikation / OCL Pre- & Postconditions / OCL Invariants

OCL PRE-CONDITIONS

```
-- all context Employee related invariants

context Employee::report_sickness(sick_days: Int)
pre PositiveSickDays:
sick_days >= 0


-- all context Caregiver related invariants
context Caregiver::submit_vacation_request_length(length: Int)
pre LengthPositive:
length > 0


-- all context Favorite_food_system/Meal-Voting related invariants

context Favorite_food_system::add_meal(mealName: String)
pre NonEmptyMealName:
not mealName.oclIsUndefined() and mealName.size() > 0


context Favorite_food_system::update_meal(mealName: String)
pre NonEmptyMealName:
not mealName.oclIsUndefined() and mealName.size() > 0


context Meal::update_votes(votes: Int)
pre VotesNonNegative:
votes >= 0

pre ValidMealId:
Meal.allInstances()->exists(m | m.meal_id = meal_id)


-- all context Child/Meal-Voting related invariants

context Child::vote_for_meal(meal_id: Int)
pre IsOldEnough:
(2025 - self.birthday_year) >= 3



-- all context Parent related invariants

context Parent::setPhoneNumber(phone: Int)
pre PhoneHas11Digits:
phone.toString().size() = 11
```

OCL POST-CONDITIONS

```
-- all context Employee related invariants

context Employee::report_sickness(sick_days: Int)
post SicknessDaysIncreased:
self.sick_days = self.sick_days@pre + sick_days



-- all context Caregiver related invariants

context Caregiver::submit_vacation_request_length(length: Int)
post VacationDaysUpdated:
self.vacation_days = self.vacation_days@pre + length



-- all context Favorite_food_system/Meal-Voting related invariants

context Favorite_food_system::add_meal(mealName: String)
post MealWasAdded:
Meal.allInstances()->exists(m | m.name = mealName and m.oclIsNew())


context Favorite_food_system::update_meal(mealName: String)
post MealWasUpdated:
Meal.allInstances()->exists(m | m.name = mealName)


context Meal::update_votes(votes: Int)
post VotesUpdated:
self.votes = votes


context Meal::set_to_zero()
post VotesReset:
self.votes = 0



-- all context Child/Meal-Voting related invariants

context Child::vote_for_meal(meal_id: Int)
post MealWasVoted:
self.votedMeal.meal_id = meal_id
```

OCL INVARIANTS

```
-- all context Employee related invariants

context Employee
inv VacationDaysWithinBounds:
self.vacation_days.oclIsInRange(0, 30)

context Employee
inv ValidID:
self.emp_id > 0

context Employee
inv SicknessDaysNonNegative:
self.sick_days >= 0

context Employee::report_sickness(sick_days: Int)
pre PositiveSickDays:
sick_days >= 0



-- all management Employee related invariants


context Management
inv CanOnlyApproveWhenTrue:
self.approve_vacation(true)



--  all caregiver Employee related invariants

context Caregiver
inv VacationLengthPositive:
self.submit_vacation_request_length(1)


context Child
inv MinimumAgeToVote:
(2025 - self.birthday_year) >= 3 implies self.vote_for_meal(0)
<> null



-- all meal-voting context related invariants

context Meal
inv VotesNonNegative:
self.votes >= 0

context Favorite_food_system::add_meal(mealName: String)
pre NonEmptyMealName:
not mealName.oclIsUndefined() and mealName.size() > 0

context Favorite_food_system
inv ValidVoteReference:
Meal.allInstances()->exists(m | m.meal_id =
self.vote_for_meal(0))



-- all daycare group related invariants
context Daycare_group
inv AtLeastOneChild:
self.children_ID->size() >= 1

context Daycare_group
inv AtLeastTwoCaregivers:
self.caregiver_ID->size() >= 2



-- all costumer context related invariants

context Costumer
inv NameNotEmpty:
not self.name.oclIsUndefined() and self.name.size() > 0

inv AddressNotEmpty:
not self.address.oclIsUndefined() and self.address.size() > 0



-- all parents context related invariants
context Parent
inv PhoneNumberHas11Digits:
self.phone_number.toString().size() = 11
```

8 Test Documentation & Quality Assurance

8.1 Code-based Testsuite

test_meal_voting_system.py

```
import unittest
from unittest.mock import patch, mock_open
import meal_voting_system as mvs
import json

class TestMealVotingSystem(unittest.TestCase):

    @patch("meal_voting_system.load_db")
    @patch("meal_voting_system.save_db")
    def test_add_meal_valid(self, mock_save, mock_load):
        mock_load.return_value = []
        response = mvs.add_meal("Pizza")
        self.assertEqual(response, "Meal added successfully")

    @patch("meal_voting_system.load_db")
    def test_add_meal_empty(self, mock_load):
        response = mvs.add_meal("")
        self.assertEqual(response, "Error: Empty name")

    @patch("meal_voting_system.load_db")
    def test_vote_invalid_meal(self, mock_load):
        mock_load.return_value = []
        response = mvs.vote_for_meal(1, 101)
        self.assertEqual(response, "Error: Meal does not exist")

    @patch("meal_voting_system.load_db")
    @patch("meal_voting_system.save_db")
    def test_update_votes_negative(self, mock_save, mock_load):
        response = mvs.update_votes(1, -5)
        self.assertEqual(response, "Error: Negative votes")

    @patch("meal_voting_system.load_db")
    @patch("meal_voting_system.save_db")
    def test_submit_vacation_request_valid(self, mock_save, mock_load):
        mock_load.return_value = []
        response = mvs.submit_vacation_request_length(101, 5)
        self.assertEqual(response, "Request submitted")

    @patch("meal_voting_system.load_db")
    def test_submit_vacation_request_invalid(self, mock_load):
        response = mvs.submit_vacation_request_length(101, 0)
        self.assertEqual(response, "Error: Minimum duration not met")

if __name__ == '__main__':
    unittest.main()
```

Code Base - neccessary function in Python

```
# meal_voting_system.py
import json
from datetime import datetime

# Load data from JSON files
def load_db(path):
    with open(path, 'r') as file:
        return json.load(file)

def save_db(data, path):
    with open(path, 'w') as file:
        json.dump(data, file, indent=2)

# Core functions - Favorite_food_system
def add_meal(name):
    if not name:
        return "Error: Empty name"
    db = load_db('db_meals.json')
    db.append({"id": len(db)+1, "name": name, "votes": 0})
    save_db(db, 'db_meals.json')
    return "Meal added successfully"

def update_meal(meal_id, new_name):
    db = load_db('db_meals.json')
    for meal in db:
        if meal['id'] == meal_id:
            meal['name'] = new_name
            save_db(db, 'db_meals.json')
            return "Meal updated"
    return "Error: Meal not found"

def remove_meal(meal_id):
    db = load_db('db_meals.json')
    updated_db = [meal for meal in db if meal['id'] != meal_id]
    if len(db) == len(updated_db):
        return "Error: Meal not found"
    save_db(updated_db, 'db_meals.json')
    return "Meal removed"

def vote_for_meal(meal_id, user_id):
    meals = load_db('db_meals.json')
    if not any(m['id'] == meal_id for m in meals):
        return "Error: Meal does not exist"
    voting = load_db('db_votings.json')
    voting.append({"meal_id": meal_id, "user_id": user_id, "timestamp":
datetime.now().isoformat()})
    save_db(voting, 'db_votings.json')
    for m in meals:
        if m['id'] == meal_id:
            m['votes'] += 1
    save_db(meals, 'db_meals.json')
    return "Vote recorded"

def update_votes(meal_id, votes):
    if votes < 0:
        return "Error: Negative votes"
    db = load_db('db_meals.json')
    for meal in db:
        if meal['id'] == meal_id:
            meal['votes'] = votes
            save_db(db, 'db_meals.json')
            return "Votes updated"
    return "Error: Meal not found"

def set_to_zero():
    meals = load_db('db_meals.json')
    for m in meals:
        m['votes'] = 0
    save_db(meals, 'db_meals.json')
    save_db([], 'db_votings.json')
    return "All votes reset"

# Scheduling_system functions
def submit_vacation_request_length(user_id, length):
    if length < 1:
        return "Error: Minimum duration not met"
    if length > 30:
        return "Error: Maximum duration exceeded"
    requests = load_db('db_requests.json')
    requests.append({"user_id": user_id, "length": length, "status": "pending"})
    save_db(requests, 'db_requests.json')
    return "Request submitted"

def approve_vacation(request_id, approve):
    if not isinstance(approve, bool):
        return "Error: Approval must be boolean"
    requests = load_db('db_requests.json')
    for req in requests:
        if req.get("id") == request_id:
            req['status'] = "approved" if approve else "rejected"
            save_db(requests, 'db_requests.json')
            return f"Vacation {'approved' if approve else 'rejected'}"
```

```
    return "Error: Request not found"

def view_shift_plan():
    return load_db('db_shifts.json')

def view_staffing_overview():
    return load_db('db_groups.json')

def alert_on_staffing_gap(threshold=4):
    groups = load_db('db_groups.json')
    alerts = [g for g in groups if len(g.get('employees', [])) <= threshold]
    return alerts if alerts else "No staffing gap detected"

def report_sickness(user_id):
    if user_id < 0:
        return "Error: Invalid user ID"
    sickness = load_db('db_sickness.json')
    sickness.append({"user_id": user_id, "timestamp": datetime.now().isoformat()})
    save_db(sickness, 'db_sickness.json')
    return "Sickness reported"

# Group and user validation
def check_groups_valid():
    groups = load_db('db_groups.json')
    for g in groups:
        if 'employee' not in g or not g['employee']:
            return False
    return True
```

8.2 Domain-based Testsuite

Function	Input	Expected Behavior	Test Class
add_meal(String)		Error: Empty name	Invalid
add_meal(String)	Spaghetti	Meal added successfully	Valid
add_meal(String)	123	Meal added successfully	Edge Case
update_meal(String)		Error: No name provided	Invalid
update_meal(String)	Pizza	Meal updated	Valid
remove_meal()	999	Error: Meal not found	Invalid
remove_meal()	3	Meal removed	Valid
vote_for_meal(Int)	-1	Error: Invalid meal ID	Invalid
vote_for_meal(Int)	3	Vote recorded	Valid
vote_for_meal(Int)	1000	Error: Meal does not exist	Edge Case
update_votes(Int)	-5	Error: Negative votes	Invalid
update_votes(Int)	0	Votes updated	Edge Case
update_votes(Int)	10	Votes updated	Valid
set_to_zero()		All votes reset	Standard
submit_vacation_request_length(Int)	-1	Error: Negative days	Invalid
submit_vacation_request_length(Int)	0	Error: Minimum duration not met	Edge Case Invalid
submit_vacation_request_length(Int)	1	Request submitted	Edge Case Valid
submit_vacation_request_length(Int)	14	Request submitted	Valid
submit_vacation_request_length(Int)	31	Error: Maximum duration exceeded	Edge Case Invalid
approve_vacation(Bool)	TRUE	Vacation approved	Valid
approve_vacation(Bool)	FALSE	Vacation rejected	Valid
approve_vacation(Bool)	yes	Error: Not a boolean	Invalid
view_shift_plan()		Plan displayed	Standard

view_staffing_overview()		Overview displayed	Standard
alert_on_staffing_gap()	3	Alert triggered	Edge Case
alert_on_staffing_gap()	10	No alert	Valid
report_sickness()	-1	Error: Invalid ID	Invalid
report_sickness()	5	Sickness reported	Valid

9 Lessons Learned

- Iterative Modellpflege verbessert Konsistenz und Verständlichkeit
- Frühzeitige Namenskonventionen erleichtern Teamarbeit
- Weitere Automatisierung (CI-Pipeline für Modell-Validierung) empfehlenswert
- Potenzial für Module: Rechnungsstellung, Eltern-Kommunikation, Analytics

10 Attachments

- 10.1 Userstories